# Windows Always On VPN — Device Tunnel Before User Logon (IKEv2, Machine-Certificate Auth) via Intune

Implementation Solution Guide

Version 1.0.0 · 2026-07-08

# Contents


Executive Summary — Windows Always On VPN — Device Tunnel Before User Logon (IKEv2, Machine-Certificate Auth) via Intune 5

- The business problem . . . 5
- The solution . . . 5
- What you get . . . 5
- Business outcomes . . . 5
- Scope and boundary . . . 6
- Bottom line . . . 6

Solution Overview — Windows Always On VPN — Device Tunnel Before User Logon (IKEv2, Machine-Certificate Auth) via Intune 6

- What this solution is . . . 6
- Why a device tunnel (and what makes it different) . . . 6
- How the device tunnel is delivered through Intune . . . 7
- Certificate lifecycle (why it is part of this solution) . . . 8
- Object naming (used verbatim everywhere) . . . 8
- Benefits . . . 8
- Limitations and boundaries . . . 9
- The standard deployment model . . . 9

Requirements — Windows Always On VPN — Device Tunnel Before User Logon (IKEv2, Machine-Certificate Auth) via Intune 10

- R1. Microsoft Intune licensing and enrolled Windows devices . . . 10
- R2. Supported Windows edition and join type (device-tunnel prerequisite) . . . 10
- R3. A VPN gateway head-end that supports the IKEv2 device tunnel . . . 10
- R4. Certificate infrastructure for machine certificates (SCEP/PKCS) . . . 11
- R5. The issuing CA's root/intermediate certificate (for the Trusted Root CA profile) . . . 11
- R6. Server certificate on the VPN gateway (head-end side) . . . 11
- R7. VPN connection parameters recorded . . . 11
- R8. Administrative roles for building the deployment . . . 12
- R9. Microsoft Graph PowerShell SDK (to run the included scripts) . . . 12
- R10. Change control and rollback (category requirement, restated) . . . 12
- Commercial vs Government (GCC / GCC High) . . . 12

Implementation — Windows Always On VPN — Device Tunnel Before User Logon (IKEv2, Machine-Certificate Auth) via Intune 13

- Object naming convention (use verbatim) . . . 13

Phase 1 — Assess 13

Phase 2 — Configure 14

2.0 Create the target DEVICE groups . . . 14
Phase 2 — Certificate lifecycle for the device tunnel (machine certificate via SCEP/PKCS) . . . 14
2.3 Author the ProfileXML . . . 16
2.4 Build the Custom OMA-URI ProfileXML profile (primary deployment path) . . . 16
Phase 3 — Pilot 17
Phase 4 — Validate 17
Phase 5 — Expand 18
Testing & Validation — Windows Always On VPN Device Tunnel (Pre-Logon) via Intune 18
Test matrix overview . . . 18
T1 — Tenant and PKI readiness . . . 19
T2 — Trusted Root CA profile applied . . . 19
T3 — Machine certificate issued . . . 19
T4 — ProfileXML applied in device context . . . 19
T5 — Device tunnel up BEFORE logon (the decisive test) . . . 20
T6 — On-device validation script . . . 20
T7 — Fleet coverage report . . . 20
T8 — Rollback / remove verification . . . 20
Overall exit criteria (before Expand) . . . 20
Troubleshooting — Windows Always On VPN Device Tunnel (Pre-Logon) via Intune 21
§Device tunnel absent — the `AOVPN Device Tunnel` profile never appears . . . 21
§Tunnel not connecting — profile present but `ConnectionStatus` is not Connected . . . 21
§Certificate issues . . . 22
§Two device tunnels — only one is allowed . . . 22
§Reporting / script issues . . . 23
Best Practices — Windows Always On VPN Device Tunnel (Pre-Logon) via Intune 23
Naming and structure . . . 23
Device context, always . . . 24
Pilot-first, always . . . 24
Certificates and the lifecycle . . . 24
ProfileXML discipline . . . 24
Change control and rollback . . . 25
Operate and monitor . . . 25
Security hygiene . . . 25
Deliverables Summary — Windows Always On VPN Device Tunnel (Pre-Logon) via Intune 25
Documentation set (source/) . . . 26
PowerShell scripts (scripts/) . . . 26

The consistent object naming convention . . . . . 27
Vendor boundary (what is deferred) . . . . . 28
Appendix A — PowerShell Scripts 29

# Executive Summary — Windows Always On VPN — Device Tunnel Before User Logon (IKEv2, Machine-Certificate Auth) via Intune

## The business problem

Remote and hybrid Windows devices need the corporate network **before a user signs in** — for a brand-new employee's first logon with no cached credentials, for an off-network password change, for group policy and logon scripts to process, and for a domain controller to be reachable at the logon screen. Without it, remote onboarding stalls, off-network password resets fail, and organizations bolt on third-party "Start Before Logon" add-ons that add cost and fragility.

## The solution

This is a complete, self-guided package that **deploys the Windows Always On VPN *device tunnel* to your managed fleet through Microsoft Intune** — the Microsoft-native way to connect the **machine** to the corporate network at **boot, before logon**. The device authenticates with its own **machine certificate** over **IKEv2**, delivered from the cloud with no hands-on device touch. It uses a disciplined **certificate-plus-ProfileXML** model (a device certificate *and* the VPNv2 ProfileXML, deployed together to the same device group) and a proven rollout method: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a tested rollback.

## What you get

- **Step-by-step implementation** at click-by-click detail on the Intune/Entra/PKI side — the machine certificate lifecycle (Trusted Root CA + SCEP device certificate), authoring the ProfileXML, and deploying it as a **Custom OMA-URI** profile in **device context**.
- **Five PowerShell scripts** — readiness discovery, on-device validation, fleet coverage reporting, safe remediation (preview-by-default), and pilot **device**-group creation.
- **Requirements, testing, troubleshooting, and best-practice guides** so a competent Intune admin can implement it end to end and prove — with a **pre-logon** test — that it works.

## Business outcomes

- **Corporate network at the logon screen** — first-time logon, off-network password changes, GPO and logon-script processing, and domain-controller reachability all work before a user is present.
- **Microsoft-native, lower cost** — the built-in device tunnel replaces third-party Start-Before-Logon add-ons; hands-off cloud delivery replaces per-device configuration.
- **Lower risk** — piloted before fleet-wide rollout, validated against explicit gates (including the

decisive pre-logon test), and reversible at any point, so a change never strands users at boot.

- **Continuous assurance** — coverage reporting shows exactly which devices have a working certificate

and device tunnel and which do not, with a renewal threshold that keeps the tunnel authenticating over time.

### Scope and boundary

The solution is authored at full detail on the **Microsoft Intune, Entra ID, and Windows/PKI** side. The **VPN gateway head-end** (Windows Server **RRAS** or **Azure VPN Gateway**) — its IKEv2 parameters, routes, CA trust, and server certificate — is the customer's existing gateway and follows **Microsoft's official documentation**. This package consumes those outputs (gateway address/FQDN, IKEv2 settings, CA trust) and delivers the certificate and ProfileXML reliably to every device.

### Bottom line

The corporate network reachable **before anyone signs in**, on every managed Windows device — delivered from Intune with a Microsoft-native IKEv2 device tunnel and machine-certificate authentication, proven on a pilot before you scale, and a fraction of the cost and time of a hands-on consulting engagement or a third-party Start-Before-Logon add-on.

## Solution Overview — Windows Always On VPN — Device Tunnel Before User Logon (IKEv2, Machine-Certificate Auth) via Intune

### What this solution is

This solution deploys the **Windows Always On VPN *device tunnel*** to your managed Windows fleet using **Microsoft Intune** — a Microsoft-native way to connect the **machine** to the corporate network at **boot, before any user signs in**. It is the built-in Windows alternative to third-party "Start Before Logon" (SBL) add-ons: the device authenticates with its own **machine certificate** over **IKEv2**, so the corporate network is reachable at the Windows logon screen — enabling first-time logon of users with **no cached credentials**, off-network **password changes**, group policy / logon-script processing, and **domain-controller reachability** before a user is present.

The work is authored at click-by-click detail on the **Intune, Entra ID, and Windows/PKI** side. The **VPN gateway head-end** (Windows Server **Routing and Remote Access (RRAS)** or **Azure VPN Gateway**) is configured **per Microsoft's official documentation** — this solution tells you *what* head-end outputs you need (gateway address/FQDN, IKEv2 settings, the root CA it trusts, the server certificate it presents) and *where* they plug into Intune, not how to click through the head-end's own console.

### Why a device tunnel (and what makes it different)

A normal Always On VPN **user tunnel** connects only *after* a user has signed in — which is useless for the very problems the device tunnel solves. The **device tunnel** is a separate, machine-context connection

with a distinct, non-negotiable shape:

- **It connects at boot, before logon.** The machine is on the corporate network at the Windows logon screen, so a brand-new user with no cached credential can authenticate against a domain controller, and an off-network user can change an expired password.
- **IKEv2 is the ONLY supported protocol.** Unlike the user tunnel, the device tunnel has **no SSTP fallback** — if IKEv2 cannot establish, the device tunnel does not connect.
- **Authentication is a machine (computer) certificate** in the device's **Local Machine** store. There is no user at boot, so there is no user credential and no user certificate — the device proves *itself* with a certificate the head-end trusts.
- **It is configured through the VPNv2 CSP's ProfileXML**, with `<DeviceTunnel>true</DeviceTunnel>` and `<AlwaysOn>true</AlwaysOn>`. **Only ONE device-tunnel profile is allowed per device.**

## How the device tunnel is delivered through Intune

There is **no app to install** — the Always On VPN device tunnel is a **native Windows VPN profile**, not a client binary. It is delivered as an **Intune *Custom* device configuration profile** that pushes the **ProfileXML** into the VPNv2 CSP in **device (SYSTEM) context**:

1. **The machine certificate (the identity).** A **SCEP** (or PKCS) certificate profile issues a **device** certificate with **Client Authentication** EKU into `LocalMachine\My`, plus the **Trusted Root CA** profile so the device trusts the issuing CA. This is delivered first — the tunnel cannot authenticate without it.
1. **The ProfileXML (the configuration).** A **Custom** profile sets the OMA-URI `./Device/Vendor/MSFT/VPNv2/AOVPN Device Tunnel/ProfileXML` to the ProfileXML string. Because it is the `./Device/` path, the profile runs in device/SYSTEM context and **must be assigned to a DEVICE group**, never a user group.

The two artifacts work together: the certificate with no ProfileXML gives the device an identity but no tunnel; the ProfileXML with no certificate gives a tunnel definition that can never authenticate. This solution always deploys **both**, assigned to the **same DEVICE group**, so a device is either fully ready or clearly not.

### *The two ways to set the profile (this solution uses the first)*

- **Custom OMA-URI ProfileXML (primary, recommended).** The full device-tunnel shape — `<DeviceTunnel>true</DeviceTunnel>`, `<AlwaysOn>true</AlwaysOn>`, routes, traffic filters, trusted network detection, and DNS (NRPT) — lives in the ProfileXML and is pushed via the Custom profile. This is the only way to set the **device-tunnel flag itself**, so it is the authoritative path.
- **Intune's built-in VPN template (supporting).** The **Windows 10 and later > Templates > VPN** profile can set **Connection type = IKEv2**, **Always On = Enable**, and **Authentication method = Machine certificates** — but the template does **not** expose the `DeviceTunnel` flag. Use the template only to understand the equivalent fields; deploy the device tunnel through the Custom OMA-URI ProfileXML.

## Certificate lifecycle (why it is part of this solution)

Because the device tunnel *requires* a valid machine certificate in `LocalMachine\My`, the **health of that certificate is the health of the tunnel**. This solution folds in a dedicated **certificate lifecycle** phase — issue the device certificate via SCEP/PKCS with the right EKU and subject/SAN, have the head-end trust the issuing CA, present a correct server certificate on the gateway, and set a **renewal threshold** that comfortably precedes expiry so the tunnel keeps authenticating over time. A pre-logon tunnel that silently stops when a certificate lapses is worse than no tunnel; the lifecycle work is what keeps it dependable.

## Object naming (used verbatim everywhere)

The names below are the exact object names you will create and reuse throughout this solution — in the steps, the tests, the troubleshooting, and the scripts.

| Object | Name |
|---|---|
| Pilot DEVICE group | `PILOT – AOVPN Device Tunnel – Devices` |
| Production DEVICE group | `PROD – AOVPN Device Tunnel – Devices` |
| Custom OMA-URI ProfileXML profile | `AOVPN – Device Tunnel – ProfileXML – Windows` |
| VPN profile name inside the XML / OMA-URI path | `AOVPN Device Tunnel` |
| Trusted Root CA profile | `AOVPN – Device Tunnel – Trusted Root CA – Windows` |
| SCEP machine certificate profile | `AOVPN – Device Tunnel – Machine Cert (SCEP) – Windows` |
| Assignment filter (optional) | `Filter – AOVPN Device Tunnel – Windows` |

## Benefits

• **Corporate network at the logon screen.** New users with no cached credential can log on off-net; off-network password changes and GPO/logon-script processing work because a domain controller is reachable before sign-in.

• **Microsoft-native, no third-party add-on.** No SBL bolt-on — the device tunnel is built into Windows and delivered from the cloud through Intune.

• **Consistent and hands-off.** Every targeted device gets the *same* ProfileXML and the *same* machine-certificate policy — configuration drift disappears.

• **Pilot-first and rollback-ready.** The certificate profiles and the ProfileXML are assigned to a named pilot **device** group, validated against explicit gates, and only then expanded. Removing the assignment (or pushing a remove profile) is the reversal.

• **Traceable and reportable.** Every object is named to a convention, so the included scripts can find and check them deterministically and report fleet coverage.

## Limitations and boundaries

• **Windows-only, and only on supported SKUs/join types.** The device tunnel is supported on devices

that are **domain-joined (or Entra hybrid-joined)** running **Windows 10/11 Enterprise or Education (version 1709 or later)**. (Support for pure Entra-joined device tunnels is emerging/edge — **verify for your build** before relying on it; do not assume it.)

• **IKEv2 only, machine-cert only, one profile.** No SSTP fallback, no user credential at boot, and

**only one device-tunnel profile per device** — a second device tunnel is not allowed.

• **VPN gateway head-end is out of scope.** The **RRAS** or **Azure VPN Gateway** configuration —

IKEv2 parameters, the address pool/routes it advertises, the root CA it trusts, and the server certificate it presents — is done **per Microsoft's official documentation**. This solution consumes those outputs (gateway address/FQDN, IKEv2 settings, CA trust) as inputs.

• **PKI is assumed.** Machine-certificate delivery depends on a working SCEP/PKCS path — on-prem

**NDES + Intune Certificate Connector** or **Intune Cloud PKI** — issuing certificates the head-end trusts.

## The standard deployment model

This solution follows the category-standard model, with explicit pass/fail gates at each step:

**Assess** → **Configure** → **Pilot** → **Validate** → **Expand**

1. **Assess.** Confirm prerequisites (Requirements guide), gather the head-end outputs (gateway

address/FQDN, IKEv2 settings, the CA it trusts), and confirm target devices are Enterprise/Education and domain-joined or hybrid-joined.

1. **Configure.** Deploy the Trusted Root CA and SCEP machine-certificate profiles, author the

ProfileXML, build the Custom OMA-URI profile, and name every object to convention.

1. **Pilot.** Assign the certificate profiles and the ProfileXML to the named pilot device group

`PILOT – AOVPN Device Tunnel – Devices` and let it deploy.

1. **Validate.** Reboot a pilot device and confirm — **at the logon screen, before any user signs

in** — the device tunnel is connected and a domain controller is reachable. Pass/fail gates in the Testing guide.

1. **Expand.** Widen assignment to `PROD – AOVPN Device Tunnel – Devices`, monitor the coverage

report, and keep the rollback (remove the assignment / push a remove profile) ready.

Proceed to the **Requirements guide** (`02-requirements.md`) to confirm you are ready to build.

# Requirements — Windows Always On VPN — Device Tunnel Before User Logon (IKEv2, Machine-Certificate Auth) via

# Intune

Confirm **every** requirement below before you build. Each entry follows the same shape: **Description / Why required / How to verify / How to implement if missing.** The category-level prerequisites (Entra tenant, Intune, admin roles, pilot group, change control) are in `global-requirements-network.md`; the items here are specific to deploying the Always On VPN device tunnel and its machine-certificate lifecycle.

> ***Vendor boundary:*** *the* ***VPN gateway head-end*** *— Windows Server* ***Routing and Remote Access (RRAS)*** *or* ***Azure VPN Gateway*** *— is configured* ***per Microsoft's official documentation****. Below, "obtain from the head-end" means *get the value from the gateway configuration your network team built per Microsoft docs* — this guide does not fabricate head-end menu paths.*

## R1. Microsoft Intune licensing and enrolled Windows devices

**Description:** An Intune-bearing license plan (e.g. Microsoft 365 E3/E5, EMS, or Intune standalone) and the target Windows devices already enrolled and shown in Intune. **Why required:** Intune is the delivery plane for the certificate profiles and the ProfileXML. Nothing deploys to a device that is not enrolled and licensed. **How to verify:** In the **Microsoft Intune admin center** (`https://intune.microsoft.com`), go to **Devices > All devices** and confirm the target devices are present and *Compliant/Managed*. Under **Tenant administration > Licenses**, confirm Intune licenses are assigned. **How to implement if missing:** Assign an Intune-bearing license (Microsoft 365 admin center → **Users** → assign licenses). Enroll Windows devices via Autopilot / Company Portal / hybrid-join enrollment.

## R2. Supported Windows edition and join type (device-tunnel prerequisite)

**Description:** The target devices run **Windows 10 or Windows 11 Enterprise or Education, version 1709 or later**, and are **domain-joined** or **Entra hybrid-joined**. **Why required:** The Always On VPN **device tunnel is supported only** on Enterprise/Education (1709+) devices that are domain-joined or hybrid-joined. Home/Pro SKUs and (currently) pure Entra-joined devices are not supported targets — the device tunnel will not deploy/connect reliably. **How to verify:** On a target device, run `winver` (edition + version) and `dsregcmd /status` (confirm `DomainJoined : YES` or `AzureAdJoined : YES` **with** `DomainJoined : YES` for hybrid). In Intune, **Devices > All devices > (device) > Hardware** shows the OS edition and join type. **How to implement if missing:** Move devices to Enterprise/Education licensing and to a domain-joined or hybrid-joined state. **Pure Entra-joined device-tunnel support is emerging/edge — verify for your build** before targeting Entra-joined-only devices; do not assume it.

## R3. A VPN gateway head-end that supports the IKEv2 device tunnel

**Description:** A reachable **RRAS** server or **Azure VPN Gateway** configured for the Always On VPN **device tunnel** — **IKEv2**, **machine-certificate** authentication, and an address pool / routes for the machine-context connection. **Why required:** The device tunnel needs something to connect *to*. The gateway address/FQDN, the IKEv2 settings, the CA it trusts, and the server certificate it presents are inputs to the ProfileXML and the certificate profiles. **How to verify:** Obtain from your network team (per Microsoft docs): the **gateway external address/FQDN** (e.g. `vpn.your-company.com`), confirmation that **IKEv2** and **machine-certificate** auth are enabled for the device tunnel, and the **root/issuing CA** the head-end trusts. **How to implement if missing:** Stand up / configure the RRAS or Azure VPN Gateway

device tunnel **per Microsoft's official documentation** — this is head-end work outside this solution's scope. Record the address/FQDN, IKEv2 settings, and CA trust for use in R4–R6 and Implementation.

## R4. Certificate infrastructure for machine certificates (SCEP/PKCS)

**Description:** A working PKI path for Intune to deliver **device (machine) certificates** — an on-prem **NDES/SCEP** connector (**Intune Certificate Connector**), **Intune Cloud PKI**, or a supported third-party CA connector. **Why required:** The device tunnel authenticates with a machine certificate in `LocalMachine\My`. There is no user at boot, so a **device** certificate must be delivered by Intune before the tunnel can connect. **How to verify:** Intune → **Devices > Configuration**; a **SCEP certificate** profile can be created and reaches devices. The NDES/Certificate Connector (if on-prem) shows healthy in **Tenant administration > Connectors and tokens**. **How to implement if missing:** Stand up the SCEP/PKCS path (NDES + Intune Certificate Connector, or enable Intune Cloud PKI). Ensure the **issuing CA is trusted by the VPN head-end** (head-end trust config is deferred to Microsoft docs).

## R5. The issuing CA's root/intermediate certificate (for the Trusted Root CA profile)

**Description:** The exported **root (and any intermediate) CA certificate** of the CA that issues the machine certificates, in `.cer` form, to deploy via Intune's **Trusted certificate** profile. **Why required:** The device must trust the chain of its own machine certificate (and the chain the head-end presents) for IKEv2 machine-cert auth to succeed. **How to verify:** You have the CA's root `.cer` file and can confirm it matches the chain the SCEP profile will issue from and the chain the head-end trusts. **How to implement if missing:** Export the root/intermediate CA certificate from your PKI, ready to upload into the `A0VPN – Device Tunnel – Trusted Root CA – Windows` profile.

## R6. Server certificate on the VPN gateway (head-end side)

**Description:** The VPN gateway presents a **server authentication** certificate whose chain the client trusts and whose subject matches the gateway address/FQDN the ProfileXML targets. **Why required:** IKEv2 mutual authentication requires the gateway to present a valid server certificate (Server Authentication EKU); a mismatch or untrusted chain breaks the tunnel. **How to verify:** Obtain from your network team confirmation that the gateway presents a server cert with **Server Authentication** EKU, a subject/SAN matching the gateway address/FQDN, and a chain the client's Trusted Root CA profile (R5) covers. **How to implement if missing:** Issue/install the gateway server certificate **per Microsoft's official documentation** (RRAS/Azure VPN Gateway). This is head-end work; this solution consumes the fact that it exists and trusts the same CA family.

## R7. VPN connection parameters recorded

**Description:** A recorded set of values for the ProfileXml: **gateway address/FQDN**, the **routes / traffic filters** the device tunnel should carry (typically only the domain-controller / management subnets), **trusted network detection** (DNS suffixes), and any **DNS (NRPT)** rules. **Why required:** These are the exact values that go inside the ProfileXML. Consistency here is what makes every device identical. Route/policy specifics on the head-end side are deferred to Microsoft docs, but the ProfileXML must reflect them. **How to verify:** You have a single sheet with each value; the routes match what the head-end advertises for the device tunnel. **How to implement if missing:** Collect the values from R3 and your network/security

owner. Decide which subnets the device tunnel carries (keep it minimal — DC/management reachability, not full tunnel) before authoring the ProfileXML.

### R8. Administrative roles for building the deployment

**Description:** Intune admin rights to create configuration profiles (Custom + certificate), and Entra rights to create/populate **device** groups (Groups Administrator or better). **Why required:** Creating the Custom OMA-URI profile, the certificate profiles, and the device groups all require elevated roles. **How to verify:** In Intune, you can reach **Devices > Configuration > Create**; in Entra you can create groups. **How to implement if missing:** Have a Global Administrator assign **Intune Administrator** and **Groups Administrator** (least-privilege) to the operator.

### R9. Microsoft Graph PowerShell SDK (to run the included scripts)

**Description:** The **Microsoft.Graph** PowerShell SDK installed on the operator workstation, used by the discovery / report / remediation / targeting scripts. **Why required:** The scripts call Microsoft Graph (Intune/Entra) to inventory the Custom profile, the certificate profiles, assignments, and device-group membership. **How to verify:** In PowerShell, `Get-Module Microsoft.Graph -ListAvailable` returns a version. **How to implement if missing:** Run `Install-Module Microsoft.Graph -Scope CurrentUser`. The scripts call `Connect-MgGraph` with the specific delegated scopes each one documents in its header. (The on-device validation script runs locally and needs **no** Graph scopes.)

### R10. Change control and rollback (category requirement, restated)

**Description:** An approved change window and a written rollback (remove the assignment / push a remove-profile for the device tunnel). **Why required:** The device tunnel touches the **pre-logon boot path** — an irreversible-feeling area for users if it misbehaves — so changes must be reversible. **How to verify:** A rollback procedure exists and is approved; you know how to unassign `PILOT – AOVPN Device Tunnel – Devices` and how to push a remove profile if needed. **How to implement if missing:** Document the rollback (Intune assignment removal; a Custom profile that removes the VPNv2 profile) and the comms plan before you assign anything.

### Commercial vs Government (GCC / GCC High)

This guide uses the **commercial** endpoints. On **US Government** tenants the portals differ — use `https://intune.microsoft.us` / `https://entra.microsoft.us` and sign-in at `login.microsoftonline.us` (and equivalent `.us` service endpoints). Feature availability and rollout timing can differ on GCC High; **verify each step for GCC High** against Microsoft's US Government documentation before relying on it. No Government-specific device-tunnel behavior is invented here.

When all requirements pass, proceed to `03-implementation.md`.

## Implementation — Windows Always On VPN — Device Tunnel Before User Logon (IKEv2, Machine-Certificate Auth) via Intune

This is the ordered, click-by-click build. Follow the phases in order: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.** Do **not** skip the pilot — the device tunnel touches the pre-logon boot path. Use the **exact object names** shown — later steps, tests, troubleshooting, and scripts reference them verbatim.

> ***Vendor boundary:*** *the VPN gateway head-end (**RRAS** or **Azure VPN Gateway**) — its IKEv2 parameters, address pool/routes, the CA it trusts, and the server certificate it presents — is **deferred to Microsoft's official documentation**. This guide covers the Intune, Entra, and Windows/PKI side at full detail and *consumes* the head-end's gateway address/FQDN, IKEv2 settings, and CA trust as inputs.*

### Object naming convention (use verbatim)

| Object | Name |
|---|---|
| Pilot DEVICE group | `PILOT – AOVPN Device Tunnel – Devices` |
| Production DEVICE group | `PROD – AOVPN Device Tunnel – Devices` |
| Custom OMA-URI ProfileXML profile | `AOVPN – Device Tunnel – ProfileXML – Windows` |
| VPN profile name inside the XML / OMA-URI path segment | `AOVPN Device Tunnel` |
| Trusted Root CA profile | `AOVPN – Device Tunnel – Trusted Root CA – Windows` |
| SCEP machine certificate profile | `AOVPN – Device Tunnel – Machine Cert (SCEP) – Windows` |
| Assignment filter (optional) | `Filter – AOVPN Device Tunnel – Windows` |

> ***The one rule that matters most:*** *the ProfileXML is pushed on the `./Device/` OMA-URI path, so it runs in **device/SYSTEM context** and **must be assigned to a DEVICE group** (`PILOT – AOVPN Device Tunnel – Devices`), **never a user group**. Assigning it to a user group is the single most common reason the device tunnel never appears (see `06-troubleshooting.md`).*

# Phase 1 — Assess

1. Confirm every item in `02-requirements.md` passes.
2. Gather the head-end inputs (from your network team, per Microsoft docs):

• **Gateway external address / FQDN** (e.g. `vpn.your-company.com`)

• Confirmation that **IKEv2** and **machine-certificate** auth are enabled for the **device tunnel**

• The **root/issuing CA** the head-end trusts, and that the gateway presents a valid **server certificate** (R6)

• The **routes / subnets** the device tunnel should carry (typically DC/management only) and any **DNS suffix / NRPT** rules (R7)

1. Confirm the pilot devices are **Enterprise/Education, 1709+, domain-joined or hybrid-joined** (R2).

2. Decide the pilot audience (3–10 **devices**, at least one that a *brand-new* user can test a

first-logon on).

1. Run the discovery script to baseline readiness:

`powershell -ExecutionPolicy Bypass -File .\scripts\AOVDT-Discovery.ps1` Review its readiness table before building anything.

**Gate A (Assess):** All requirements pass, all head-end inputs recorded, pilot devices confirmed Enterprise/Education + domain/hybrid-joined, and discovery shows the tenant is ready (PKI/connector healthy). If not, stop and remediate.

# Phase 2 — Configure

## 2.0 Create the target DEVICE groups

Use the script (recommended) or the portal.

- **Script:** `powershell -ExecutionPolicy Bypass -File .\scripts\AOVDT-Set-ScopeTargeting.ps1 -PilotDevices DEVICE01,DEVICE02`

(creates `PILOT – AOVPN Device Tunnel – Devices` and adds the **device** objects you name).

- **Portal:** Entra admin center (`https://entra.microsoft.com`) → **Identity > Groups > All groups >

New group → Group type: Security → Group name: `PILOT – AOVPN Device Tunnel – Devices` → **add the pilot** devices **under** Members **(search device display names)** → Create**. Repeat for `PROD – AOVPN Device Tunnel – Devices`.

> *These are **device** groups (members are devices, not users), because the ProfileXML deploys in device context.*

## Phase 2 — Certificate lifecycle for the device tunnel (machine certificate via SCEP/PKCS)

**Do this before the ProfileXML.** The device tunnel authenticates with a **machine certificate in** `LocalMachine\My` — there is no user at boot, so a **device** certificate must already exist when the tunnel tries to connect. If the certificate is missing, the wrong EKU, or issued by a CA the head-end does not trust, the tunnel will never authenticate no matter how correct the ProfileXML is.

### *2.1 Deploy the Trusted Root CA profile (first)*

1. Intune admin center → **Devices > Configuration > Create > New Policy**.

2. **Platform:** *Windows 10 and later*. **Profile type:** *Templates* → **Trusted certificate** →

**Create**.

1. **Name:** `AOVPN – Device Tunnel – Trusted Root CA – Windows`. Add a description. **Next.**

2. **Configuration settings:** upload the **root (and intermediate, as a second profile if needed)** CA `.cer` from R5; **Destination store:** *Computer certificate store – Root* (for intermediates, *Computer certificate store – Intermediate*). **Next.**

1. **Assignments:** add `PILOT – AOVPN Device Tunnel – Devices`. **Next > Review + create > Create.**

*Deploy the trusted root **before** the SCEP profile so the device already trusts the chain when its machine certificate is issued.*

### *2.2 Deploy the SCEP machine-certificate profile*

1. Intune admin center → **Devices > Configuration > Create > New Policy**.

2. **Platform:** *Windows 10 and later*. **Profile type:** *Templates* → **SCEP certificate** → **Create**. (Use **PKCS certificate** instead if your PKI uses PKCS — the field intent is the same.)

1. **Name:** `AOVPN – Device Tunnel – Machine Cert (SCEP) – Windows`. Add a description. **Next.**

2. **Configuration settings:**

• **Certificate type: Device** (this is what puts the certificate in `LocalMachine\My` with no user — critical for the device tunnel).

• **Subject name format:** e.g. `CN={{DeviceName}}` (or the format your head-end matches on).

• **Subject alternative name:** add a **DNS** SAN of `{{FullyQualifiedDomainName}}` (or the value your head-end expects).

• **Certificate validity period:** per your PKI policy.

• **Key storage provider (KSP):** *Enroll to Trusted Platform Module (TPM) KSP if present, otherwise Software KSP*.

• **Key usage:** *Digital signature* and *Key encipherment*.

• **Extended key usage:** add **Client Authentication** (`1.3.6.1.5.5.7.3.2`) — the device tunnel requires it.

• **Root Certificate:** click **Root Certificate**, choose the `AOVPN – Device Tunnel – Trusted Root CA – Windows` profile from 2.1.

• **SCEP Server URLs:** the SCEP/NDES (or Cloud PKI) enrollment URL(s) from R4.

• **Renewal threshold (%):** set comfortably ahead of expiry (e.g. **20%**) so the certificate renews **before** it lapses and the tunnel never loses authentication.

1. **Assignments:** add `PILOT – AOVPN Device Tunnel – Devices`. **Next > Review + create > Create.**

***Head-end trust (deferred to Microsoft docs):** the VPN gateway must **trust the issuing CA** and **present its own server certificate** (R6) with **Server Authentication** EKU whose subject matches the gateway address/FQDN. That trust/server-cert configuration is done on the RRAS / Azure VPN*

*Gateway side **per Microsoft's official documentation** — this solution ensures the device holds the right client certificate and trusts the right root.*

**Gate B-cert (Certificate):** On a pilot device, `AOVPN – Device Tunnel – Trusted Root CA – Windows` and `AOVPN – Device Tunnel – Machine Cert (SCEP) – Windows` both show *Succeeded* in Intune, and a certificate with **Client Authentication** EKU is present in `Cert:\LocalMachine\My`. If not, resolve before deploying the ProfileXML.

## 2.3 Author the ProfileXML

Author the device-tunnel **ProfileXML** (per Microsoft's Always On VPN ProfileXML reference). The device-tunnel-defining elements are:

• `<AlwaysOn>true</AlwaysOn>` and `<DeviceTunnel>true</DeviceTunnel>` — this is what makes it the

boot-time device tunnel (the Intune VPN *template* cannot set `DeviceTunnel`).

• `<NativeProfile>` with `<Servers>` = the gateway address/FQDN (R7) and

`<NativeProtocolType>IKEv2</NativeProtocolType>` — **IKEv2 only, no SSTP fallback**.

• `<Authentication>` = **machine certificate** (the device presents its `LocalMachine\My` cert; no

user credential).

• **Routes / traffic filters** — the subnets the device tunnel carries (keep minimal: DC/management),

and where **trusted network detection** (`<TrustedNetworkDetection>` DNS suffixes) and **DNS (NRPT)** rules go. The exact route values come from R7 / the head-end (deferred to Microsoft docs); the ProfileXML is *where they live*.

Keep the authored XML on a single line (or escape it) for the OMA-URI value in 2.4. The **VPN profile name inside the XML / OMA-URI path segment is** `AOVPN Device Tunnel` — use it verbatim.

## 2.4 Build the Custom OMA-URI ProfileXML profile (primary deployment path)

1. Intune admin center → **Devices > Configuration > Create > New Policy**.
2. **Platform:** *Windows 10 and later*. **Profile type:** *Templates* → **Custom** → **Create**.
3. **Name:** `AOVPN – Device Tunnel – ProfileXML – Windows`. Add a description. **Next.**
4. **Configuration settings** → **Add** an OMA-URI setting:

• **Name:** `AOVPN Device Tunnel ProfileXML`

• **OMA-URI:** `./Device/Vendor/MSFT/VPNv2/AOVPN Device Tunnel/ProfileXML`

(the path segment `AOVPN Device Tunnel` is the VPN profile name; keep it identical everywhere).

• **Data type: String**.

• **Value:** paste the ProfileXML authored in 2.3.

• **Save.**

1. **Assignments:** add `PILOT – AOVPN Device Tunnel – Devices` (a **DEVICE** group — because the

OMA-URI is on the `./Device/` path). Optionally attach `Filter – AOVPN Device Tunnel – Windows`. **Next > Review + create > Create.**

> ***Do not*** *assign this Custom profile to a user group. The `./Device/` path is device/SYSTEM context; a user-group assignment silently does nothing at the machine level, and the device tunnel never appears.*

### *(Reference only) The equivalent built-in VPN template fields*

Intune's **Windows 10 and later > Templates > VPN** profile can set **Connection type = IKEv2**, **Always On = Enable**, and **Authentication method = Machine certificates** — useful to understand the equivalent settings — but it **cannot set the** `DeviceTunnel` **flag**. Deploy the device tunnel through the Custom OMA-URI ProfileXML above; the template is not a substitute.

**Gate B (Configure):** The Trusted Root CA profile, the SCEP machine-certificate profile, and the Custom OMA-URI ProfileXML profile all exist, are named to convention, and are assigned to `PILOT – AOVPN Device Tunnel – Devices`. Re-run `AOVDT-Discovery.ps1` — every expected object should now be found and assigned to the **device** group.

## Phase 3 — Pilot

1. Ensure the pilot **devices** are enrolled, compliant, Enterprise/Education + domain/hybrid-joined,

and members of `PILOT – AOVPN Device Tunnel – Devices`.

1. Trigger a sync (Intune → **Devices > (device) > Sync**, or on the device **Settings > Accounts >

Access work or school > Info > Sync**) to pull the certificate profiles and the ProfileXML.

1. Allow time for the certificate to issue (`LocalMachine\My`) and the VPNv2 profile to apply.
2. **Reboot** the pilot device so the device tunnel establishes at boot.

**Gate C (Pilot):** Every pilot device shows the two certificate profiles and the Custom ProfileXML profile *Succeeded* in Intune monitoring, and the machine certificate is present. If any fail, go to `06-troubleshooting.md` before proceeding.

## Phase 4 — Validate

1. On a pilot device, run:

`powershell -ExecutionPolicy Bypass -File .\scripts\AOVDT-Validation.ps1` It confirms the `AOVPN Device Tunnel` profile is present and a valid machine certificate exists on *this* device.

1. Perform the **decisive pre-logon test** in `05-testing.md`: **reboot the device and, at the Windows

logon screen with no user signed in**, confirm the device tunnel is up and a domain controller is reachable (e.g. `Get-VpnConnection -AllUserConnection` shows `AOVPN Device Tunnel` *Connected*, and

a DC/internal host is reachable before any user logs on).

1. Run the fleet report to see pilot coverage:

```
powershell -ExecutionPolicy Bypass -File .\scripts\AOVDT-Report.ps1
```

**Gate D (Validate):** On every pilot device, both certificate profiles and the ProfileXML apply, and the **device tunnel is Connected at the logon screen before user sign-in** with a domain controller reachable. All Testing guide pass/fail gates are green. Only then proceed.

## Phase 5 — Expand

1. For the two certificate profiles and the Custom ProfileXML profile, **add**

`PROD – AOVPN Device Tunnel – Devices` to the **Assignments**. Keep `PILOT – AOVPN Device Tunnel – Devices` assigned (pilot devices stay covered).

1. Expand in waves if the fleet is large (add production sub-groups of devices over several days) and

watch the report between waves.

1. Monitor with `AOVDT-Report.ps1` for coverage and with Intune **Devices > Configuration** for

profile success.

1. Remediate stragglers with `AOVDT-Remediation.ps1` (defaults to **-WhatIf** preview — review, then

re-run with `-WhatIf:$false` to act).

**Rollback (any phase):** Remove `PILOT/PROD – AOVPN Device Tunnel – Devices` from the ProfileXML and certificate profile **Assignments**; to actively remove the tunnel, push a Custom profile that removes the `AOVPN Device Tunnel` VPNv2 profile. Because it is a pre-logon path, test the rollback on a disposable device first (Testing T8).

**Gate E (Expand):** Fleet coverage meets your target (e.g. ≥ 95% of assigned devices with both certificate profiles and the ProfileXML *Succeeded* and the device tunnel connecting pre-logon); remaining failures are triaged. Deployment is complete.

# Testing & Validation — Windows Always On VPN Device Tunnel (Pre-Logon) via Intune

Run these tests on the **pilot** before expanding, and spot-check during expansion. Each test has an explicit **pass/fail gate**. A phase is only "done" when its gate is **PASS**. Object names match the Implementation guide verbatim.

> ***Scope note:*** *these tests validate that Intune* ***delivered the certificate and the ProfileXML*** *and that a device* ***establishes the device tunnel at boot, before logon****. Deep head-end diagnostics (why the gateway rejects a specific certificate, IKEv2 negotiation internals) are head-end-side and deferred to Microsoft docs.*

### Test matrix overview

| # | Test | Where | Pass gate |
|---|---|---|---|
| T1 | Tenant / PKI readiness | Discovery script | Objects found; PKI healthy |
| T2 | Trusted Root CA profile applied | Intune monitor + device | Profile *Succeeded*; root trusted |
| T3 | Machine certificate issued | Device | Cert in `LocalMachine\My` with Client Auth EKU |
| T4 | ProfileXML applied (device context) | Intune monitor + device | Custom profile *Succeeded*; `AOVPN Device Tunnel` present |
| T5 | **Device tunnel up BEFORE logon** | Pilot device at logon screen | Tunnel *Connected*; DC reachable pre-logon |
| T6 | On-device validation script | Pilot device | Script reports PASS |
| T7 | Fleet coverage report | Report script | Coverage ≥ target |
| T8 | Rollback / remove | Test device | Profile + tunnel removed cleanly |

## T1 — Tenant and PKI readiness

**Steps:** Run `AOVDT-Discovery.ps1`. Review the readiness table: the Trusted Root CA profile, the SCEP machine-cert profile, and the Custom ProfileXML profile found and assigned to `PILOT – AOVPN Device Tunnel – Devices`, and the pilot/prod **device** groups present. **PASS:** Every expected object is present and assigned to the **device** group; PKI/connector healthy. **FAIL:** Any object missing/unassigned, or assigned to a **user** group → fix in Implementation before piloting.

## T2 — Trusted Root CA profile applied

**Steps:** Intune → **Devices > Configuration >** `AOVPN – Device Tunnel – Trusted Root CA – Windows` **> Device status**. On the device, open `certlm.msc` → **Trusted Root Certification Authorities** and confirm the issuing CA's root is present. **PASS:** Profile *Succeeded* and the root CA is in the machine Trusted Root store. **FAIL:** *Error* or root absent → see Troubleshooting §Certificate issues.

## T3 — Machine certificate issued

**Steps:** Intune → **Devices > Configuration >** `AOVPN – Device Tunnel – Machine Cert (SCEP) – Windows` **> Device status**. On the device, run `Get-ChildItem Cert:\LocalMachine\My | Where-Object { $_.EnhancedKeyUsageList.FriendlyName -match 'Client Authentication' }`. **PASS:** SCEP profile *Succeeded*; a non-expired certificate with **Client Authentication** EKU is in `LocalMachine\My` and chains to a CA the head-end trusts. **FAIL:** No cert / wrong EKU / wrong chain → see Troubleshooting §Certificate issues.

## T4 — ProfileXML applied in device context

**Steps:** Intune → **Devices > Configuration >** `AOVPN – Device Tunnel – ProfileXML – Windows` **> Device status**. On the device, run `Get-VpnConnection -AllUserConnection` and confirm the `AOVPN Device Tunnel` profile is listed. **PASS:** Custom profile *Succeeded* and the `AOVPN Device Tunnel` profile exists **at the machine level** (`-AllUserConnection`). **FAIL:** *Error*/*Conflict*, or the profile appears only per-user / not at all → confirm the profile is assigned to the **DEVICE** group (see Troubleshooting §Device tunnel absent).

## T5 — Device tunnel up BEFORE logon (the decisive test)

**Steps: Reboot** the pilot device. **At the Windows logon screen, with no user signed in**, confirm the device tunnel is established — e.g. from another session or a management path, or by signing in as a **brand-new** user with **no cached credential** to confirm authentication against a domain controller succeeds off-network. Confirm `Get-VpnConnection -AllUserConnection` reports `AOVPN Device Tunnel` with `ConnectionStatus : Connected`, and an internal DC/host is reachable that is *not* reachable without the tunnel. **PASS:** The device tunnel is *Connected* at/around the logon screen before any interactive user signs in, and a domain controller / internal resource is reachable pre-logon. **FAIL:** Tunnel not connected pre-logon, or DC unreachable → see Troubleshooting §Tunnel not connecting (and confirm head-end/IKEv2/CA trust per Microsoft docs).

## T6 — On-device validation script

**Steps:** On a pilot device run `AOVDT-Validation.ps1`. **PASS:** Script reports the `AOVPN Device Tunnel` profile present **and** a valid machine certificate in `LocalMachine\My` → overall **PASS**. **FAIL:** Script reports a missing component → address per its message before counting the device.

## T7 — Fleet coverage report

**Steps:** Run `AOVDT-Report.ps1`; review certificate + ProfileXML apply coverage across the assigned device groups. **PASS (pilot):** 100% of pilot devices with both cert profiles and the ProfileXML *Succeeded*. **PASS (expand):** Coverage ≥ your target (e.g. ≥ 95%) with remaining failures triaged. **FAIL:** Coverage below target → remediate (T2/T3/T4 failures) before widening further.

## T8 — Rollback / remove verification

**Steps:** On a disposable test device, remove it from `PILOT – AOVPN Device Tunnel – Devices` (or push a Custom profile that removes the `AOVPN Device Tunnel` VPNv2 profile). Sync and reboot. **PASS:** The `AOVPN Device Tunnel` profile is removed; the device boots normally and users log on without the device tunnel. No pre-logon disruption. **FAIL:** Residual profile or a broken logon path → confirm the remove profile / unassignment took effect before touching production.

## Overall exit criteria (before Expand)

All of T1–T6 **PASS** on every pilot device, **T5 confirms the tunnel is up before logon with a DC reachable**, T7 shows 100% pilot coverage, and T8 confirms a clean rollback. Record results against each gate; only then proceed to Phase 5 (Expand).

# Troubleshooting — Windows Always On VPN Device Tunnel (Pre-Logon) via Intune

Common failures and fixes, grouped by symptom. Object names match the Implementation guide. Work top-down within a section — the first cause listed is the most common.

> ***Vendor boundary:*** *problems on the* ***VPN gateway head-end*** *(IKEv2 negotiation fails, the gateway rejects the certificate, the gateway's own server certificate/CA trust) are* ***deferred to Microsoft's official documentation and your network team****. This guide resolves the* ***Intune / Entra / Windows / PKI delivery*** *side and tells you when a failure points at the head-end.*

## §Device tunnel absent — the `AOVPN Device Tunnel` profile never appears

**Symptom:** After sync/reboot, `Get-VpnConnection -AllUserConnection` does not list `AOVPN Device Tunnel`, and Intune may still show the Custom profile *Succeeded* (or in error).

1. **Profile assigned to a USER group, not a DEVICE group.** This is the single most common cause. The

OMA-URI is on the `./Device/` path (SYSTEM/device context), so `AOVPN – Device Tunnel – ProfileXML – Windows` **must** be assigned to a **device** group (`PILOT – AOVPN Device Tunnel – Devices`). A user-group assignment silently does nothing at the machine level. Re-assign to the device group (`AOVDT-Discovery.ps1` shows the assignment target; `AOVDT-Remediation.ps1 -FixMissingAssignments` adds the device-group assignment).

1. **Device not domain-joined / hybrid-joined, or wrong SKU.** The device tunnel is supported only on

**Enterprise/Education (1709+)** that is **domain-joined or Entra hybrid-joined** (R2). Confirm with `winver` and `dsregcmd /status`. Pure Entra-joined device-tunnel support is emerging/edge — **verify for your build**; it may simply not apply.

1. **OMA-URI or profile name mismatch.** The OMA-URI must be exactly

`./Device/Vendor/MSFT/VPNv2/AOVPN Device Tunnel/ProfileXML` and the profile name inside the XML must match `AOVPN Device Tunnel`. A mismatched name produces a profile you can't find by name.

1. **Device not synced.** Force a sync (Intune → device → **Sync**) and **reboot** — the device tunnel

establishes at boot. Check the Intune Management Extension / MDM diagnostics if it persists.

## §Tunnel not connecting — profile present but `ConnectionStatus` is not Connected

**Symptom:** `AOVPN Device Tunnel` is listed by `Get-VpnConnection -AllUserConnection` but does not connect at boot / stays *Disconnected*.

1. **Machine certificate missing or wrong.** The device tunnel authenticates with a **machine

certificate **in** `LocalMachine\My` **with** Client Authentication **EKU. Run** `AOVDT-Validation.ps1`**; if the certificate is absent or the EKU is wrong, fix the SCEP profile (**`AOVPN – Device Tunnel –`

`Machine Cert (SCEP) – Windows`**) —** Certificate type = Device**, EKU =** Client Authentication** — and re-sync. See §Certificate issues.

1. **Head-end doesn't trust the issuing CA (head-end side).** If the client cert is correct but IKEv2

still fails auth, the **gateway must trust the issuing CA** — this trust config is **head-end-side, deferred to Microsoft docs**. Provide your network team the issuing CA chain.

1. **Gateway server certificate / FQDN mismatch (head-end side).** The gateway must present a **Server

Authentication** certificate whose subject matches the address/FQDN in the ProfileXML, chaining to a CA the device trusts (R5/R6). A mismatch or untrusted chain breaks IKEv2.

1. **IKEv2 not reachable.** There is **no SSTP fallback** — if IKEv2 (UDP 500/4500 path to the gateway)

is blocked from the network the device boots on, the device tunnel simply won't come up. Confirm the path with your network team.

1. **Wrong gateway address/FQDN in the ProfileXML.** Verify `<Servers>` matches the head-end

address/FQDN from R7 and the device can resolve/reach it.

## §Certificate issues

**Symptom:** `AOVPN – Device Tunnel – Machine Cert (SCEP) – Windows` fails, or the tunnel rejects the certificate.

1. **Trusted root not deployed first.** The device must trust the issuing CA before the SCEP profile

succeeds. Deploy `AOVPN – Device Tunnel – Trusted Root CA – Windows` **first**, then the SCEP profile.

1. **Certificate type is User, not Device.** A user certificate is useless at boot (no user). The SCEP

profile's **Certificate type** must be **Device** so the cert lands in `LocalMachine\My`.

1. **Missing Client Authentication EKU.** The device tunnel requires **Client Authentication**

(`1.3.6.1.5.5.7.3.2`) in the SCEP profile's EKU list. Add it and re-issue.

1. **SCEP infrastructure error.** Check NDES / Intune Certificate Connector health and the SCEP server

URL(s) in the profile. Re-sync after fixing.

1. **Subject/SAN mismatch.** If the head-end matches on subject/SAN, ensure the SCEP profile's

subject-name format and SAN produce what the head-end expects (coordinate the required format with your network team).

1. **Certificate expired (renewal too late).** If the tunnel worked and then stopped, the machine

certificate may have lapsed. Set a comfortable **renewal threshold** (e.g. 20%) so it renews before expiry, and re-issue.

## §Two device tunnels — only one is allowed

**Symptom:** Conflicts / unpredictable device-tunnel behavior; a second ProfileXML also sets `DeviceTunnel=true`.

1. **Windows allows only ONE device-tunnel profile per device.** Deploying two Custom profiles that each set `<DeviceTunnel>true</DeviceTunnel>` conflicts. Keep exactly one (`AOVPN – Device Tunnel – ProfileXML – Windows`); remove/scope any other device-tunnel ProfileXML. Use pilot vs prod **device** groups to stage — never duplicate the profile.

## §Reporting / script issues

1. `Connect-MgGraph` **fails or lacks scopes.** Install the SDK (`Install-Module Microsoft.Graph -Scope CurrentUser`) and consent to the scopes each script lists in its header. Read-only scripts need only `.Read.All` scopes. (`AOVDT-Validation.ps1`) is local and needs **no** Graph scopes.)

1. **Objects show MISSING but exist in the portal.** The scripts match **by exact display name** — confirm the object is named to convention verbatim (including the en-dash – and spacing). Rename any drifted object to the convention.

1. **Discovery shows the ProfileXML assigned to a user group.** That is a real problem, not a script bug — re-assign to `PILOT – AOVPN Device Tunnel – Devices` (see §Device tunnel absent #1).

1. **Remediation "made no changes."** By design — it defaults to **-WhatIf**. Re-run with `-WhatIf:$false` after reviewing the previewed actions.

If a failure persists after the Intune/Entra/PKI side is confirmed healthy (validation PASS, both certificate profiles *Succeeded*, machine cert present with Client Authentication EKU, ProfileXML applied in device context) and the tunnel still won't establish or authenticate, treat it as a **VPN gateway head-end issue** and continue with **Microsoft's official documentation** (RRAS / Azure VPN Gateway) and your network team.

# Best Practices — Windows Always On VPN Device Tunnel (Pre-Logon) via Intune

Field-proven practices for the assign-once, repeat-everywhere model. These make the device-tunnel deployment predictable, reversible, and safe to operate at scale — especially because it touches the pre-logon boot path.

## Naming and structure

- **Name every object to the convention and reuse it verbatim.** The two certificate profiles, the Custom ProfileXML profile, the device groups, and the filter all follow `AOVPN – Device Tunnel – …` / `PILOT – AOVPN Device Tunnel – Devices`. The discovery, validation, report, and remediation scripts find objects **by these names** — drift breaks automation and reporting.

- **Keep the VPN profile name identical everywhere.** The VPNv2 profile name is `AOVPN Device Tunnel` in the OMA-URI path, inside the ProfileXML, and in every `Get-VpnConnection` check. A mismatch here produces a profile that looks deployed but can't be found by name.

- **One device-tunnel profile per device — only.** Windows allows **only one** device-tunnel profile.

Never deploy two ProfileXML Custom profiles that both set `DeviceTunnel=true`; use assignment (pilot vs prod device group) to stage, not duplicate profiles.

## Device context, always

- **Assign the ProfileXML to a DEVICE group, never a user group.** The `./Device/` OMA-URI runs in SYSTEM/device context; a user-group assignment silently does nothing. This is the single most common device-tunnel mistake — make the device-group assignment a checklist item.
- **Use device groups for the certificate profiles too.** The machine certificate is a **Device**-type certificate in `LocalMachine\My`; target the same device groups so cert and ProfileXML land on the same machines.

## Pilot-first, always

- **Never skip the pilot,** even for a "small" ProfileXML change. Every change to the ProfileXML or the certificate policy goes to `PILOT – AOVPN Device Tunnel – Devices` first and passes the Testing gates — including the **pre-logon** test — before production.
- **Pilot with a first-logon device.** Include at least one device where a *brand-new* user with **no cached credential** logs on, to prove the whole point of the device tunnel actually works.
- **Expand in waves.** For large fleets, add production device sub-groups over several days and watch `AOVDT-Report.ps1` between waves rather than assigning everyone at once.

## Certificates and the lifecycle

- **Deploy the trusted root first, then SCEP, then the ProfileXML** — order matters so the machine certificate exists and chains correctly before the tunnel needs it.
- **Issue a Device certificate with Client Authentication EKU.** The device tunnel has no user at boot; a User-type certificate or a missing Client Authentication EKU will not authenticate.
- **Set a comfortable SCEP renewal threshold** (e.g. 20%) so certificates renew **before** expiry — a lapsed machine certificate silently kills the pre-logon tunnel. Watch cert profile status in the report.
- **Ensure the head-end trusts the issuing CA and presents a valid server certificate** (Server Authentication EKU, subject matching the gateway FQDN). This head-end trust config is deferred to Microsoft docs, but it is the other half of IKEv2 mutual auth — provide your network team the issuing CA chain.

## ProfileXML discipline

- **Keep the device tunnel minimal.** Carry only the routes the machine needs pre-logon (domain

controllers, management/PKI subnets) — not a full tunnel. Over-broad routing hurts performance and is unnecessary for the device tunnel's job.

• **IKEv2 only — plan for it.** There is no SSTP fallback; ensure the IKEv2 ports/path to the gateway are reachable from the networks your devices boot on, or the device tunnel simply won't come up.

• **Version your ProfileXML.** Track which ProfileXML (gateway, routes, NRPT) and which certificate policy a rollout used, so you can reproduce or revert precisely.

### Change control and rollback

• **Keep a one-click rollback ready:** remove the device-group assignment, or push a Custom profile that removes the `AOVPN Device Tunnel` VPNv2 profile. Test the rollback (Testing T8) on a disposable device **before** you ever need it — this is a pre-logon path.

• **Deploy inside a change window** and announce it — a misbehaving device tunnel affects boot/logon for real users.

### Operate and monitor

• **Run** `AOVDT-Report.ps1` **on a schedule** (e.g. weekly) to track coverage drift as devices enroll and leave, and to catch profile *Error* / certificate-expiry states early.

• **Remediate safely.** `AOVDT-Remediation.ps1` defaults to **-WhatIf** — always preview the planned actions, then re-run with `-WhatIf:$false`. Never let a remediation run act without a preview first.

• **Least-privilege admin.** Use Intune Administrator + Groups Administrator for the operator; reserve Global Administrator for tenant-wide changes only.

### Security hygiene

• **No credentials in the profile.** The device tunnel authenticates with a machine certificate only — never embed secrets in the ProfileXML.

• **Protect the machine key.** Prefer TPM-backed key storage for the SCEP certificate so the device identity can't be trivially exported.

• **Scope routes deliberately.** Only tunnel the internal subnets the machine genuinely needs before logon; the user tunnel (a separate deployment) handles user-context traffic.

## Deliverables Summary — Windows Always On VPN Device Tunnel (Pre-Logon) via Intune

This solution is a complete, self-guided package to deploy the Windows **Always On VPN device tunnel** — machine-context, IKEv2, machine-certificate authentication, connecting **before user logon** — to your managed Windows fleet using Microsoft Intune, pilot-first and rollback-ready. Below is everything included.

## Documentation set (source/)

| Document | What it gives you |
|---|---|
| `01-solution-overview.md` | Why a device tunnel (pre-logon, IKEv2-only, machine-cert-only, one profile), how it's delivered via a Custom OMA-URI ProfileXML in device context, the folded-in certificate lifecycle, benefits, limits, and the Assess→Configure→Pilot→Validate→Expand model. |
| `02-requirements.md` | Every prerequisite with **Description / Why / Verify / Implement-if-missing** — Intune licensing + enrollment, supported Windows edition/join type, the VPN head-end, SCEP/PKCS PKI, the root CA and gateway server cert, connection parameters, admin roles — plus the Commercial vs Government (GCC High) note. |
| `03-implementation.md` | Exact ordered build: create the DEVICE groups, **the certificate lifecycle phase** (Trusted Root CA then SCEP machine cert), author the ProfileXML, build the Custom OMA-URI profile assigned to a DEVICE group, then pilot, validate (pre-logon), and expand — with Gates A–E. |
| `05-testing.md` | The validation test matrix (T1–T8) with explicit **PASS/FAIL** gates, including the decisive **pre-logon** device-tunnel test. |
| `06-troubleshooting.md` | Failure-by-symptom fixes: device tunnel absent, tunnel not connecting, certificate issues, two device tunnels, and script/reporting issues. |
| `07-best-practices.md` | Field-proven practices for naming, device-context assignment, certificate lifecycle, ProfileXML discipline, change control, and safe operation. |
| `executive-summary.md` | One-page business-outcome summary for stakeholders. |
| `global-requirements-network.md` | Category-level Network prerequisites (shared across Network solutions). |

## PowerShell scripts (scripts/)

All five use the **Microsoft Graph PowerShell SDK** except the on-device validation script, which runs locally with **no** Graph scopes. Discovery/validation/report are **read-only**; remediation and scope-targeting **default to** `-WhatIf` **(preview)** and change nothing until you re-run with `-WhatIf:$false`. Each script's header documents its exact purpose, run command, and required scopes.

| Script | Purpose | Mode |
|---|---|---|
| `AOVDT-Discovery.ps1` | Deployment-readiness sweep across Entra + Intune — finds the Custom ProfileXML profile, the Trusted Root CA and SCEP cert profiles, and the pilot/prod **device** groups, with assignment state. (There is **no app** — the device tunnel is a native VPN profile.) | Read-only |
| `AOVDT-Validation.ps1` | Confirms **on this device** that the `AOVPN Device Tunnel` profile is present (`Get-VpnConnection -AllUserConnection`) and a valid machine certificate with Client Authentication EKU is in `LocalMachine\My`. Prints an overall PASS/FAIL. | Read-only (local) |
| `AOVDT-Report.ps1` | Fleet coverage report — per-profile apply counts (succeeded/failed/pending) for the ProfileXML and certificate profiles, with coverage percentages and an overall figure; optional CSV export. | Read-only |
| `AOVDT-Remediation.ps1` | Safely fixes stragglers — retries (device sync) devices where the ProfileXML errored and adds missing Required assignments to the target **device** group. | Write, **default** `-WhatIf` |
| `AOVDT-Set-ScopeTargeting.ps1` | Creates/populates the pilot (and optional production) Entra **device** security groups used as assignment targets — adding **devices** by name. | Write, **default** `-WhatIf` |

## The consistent object naming convention

Every artifact is named to a convention and reused verbatim across docs, tests, troubleshooting, and scripts (so the scripts find objects deterministically):

- Device groups: `PILOT – AOVPN Device Tunnel – Devices`, `PROD – AOVPN Device Tunnel – Devices`
- Custom ProfileXML profile: `AOVPN – Device Tunnel – ProfileXML – Windows`

(VPN profile name inside the XML / OMA-URI path segment: `AOVPN Device Tunnel`)

- Trusted Root CA profile: `AOVPN – Device Tunnel – Trusted Root CA – Windows`

- SCEP machine certificate profile: `AOVPN – Device Tunnel – Machine Cert (SCEP) – Windows`
- Assignment filter (optional): `Filter – AOVPN Device Tunnel – Windows`

## Vendor boundary (what is deferred)

The **VPN gateway head-end** — Windows Server **RRAS** or **Azure VPN Gateway** — including its IKEv2 parameters, address pool/routes, the CA it trusts, and the server certificate it presents, is **deferred to Microsoft's official documentation**. This solution authors the **Intune, Entra, and Windows/PKI** side at full click-by-click detail and consumes the head-end's gateway address/FQDN, IKEv2 settings, and CA trust as inputs.

# Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

## AOVDT-Discovery.ps1

**Purpose:** Read-only readiness sweep for a Windows Always On VPN device-tunnel deployment via Intune. Finds the Custom OMA-URI ProfileXML configuration profile ('AOVPN – Device Tunnel – ProfileXML – Windows', a windows10CustomConfiguration with an omaSettings entry), the Trusted Root CA and SCEP machine-certificate profiles, and the pilot/production DEVICE groups, and reports which expected objects are present, named to convention, and assigned. Makes NO changes. NOTE: there is NO app to find — the device tunnel is a native Windows VPN profile (VPNv2 CSP), delivered as a Custom configuration profile, not an installed application. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AOVDT-Discovery.ps1 Required Microsoft Graph scopes (delegated, read-only): DeviceManagementConfiguration.Read.All DeviceManagementServiceConfig.Read.All Group.Read.All Directory.Read.All EXPECTED RESULTS A readiness table: the Custom ProfileXML profile, the Trusted Root CA profile, and the SCEP machine-cert profile each Found/Missing with assignment state (and whether the ProfileXML is assigned to a DEVICE group), plus the pilot device-group membership count. All green = ready to pilot; any Missing/Unassigned = fix before deploying (see 02-requirements.md / 03-implementation.md). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. VPN gateway head-end internals deferred to vendor docs. >

**How to run:** powershell -ExecutionPolicy Bypass -File .\AOVDT-Discovery.ps1 Required Microsoft Graph scopes (delegated, read-only): DeviceManagementConfiguration.Read.All DeviceManagementServiceConfig.Read.All Group.Read.All Directory.Read.All EXPECTED RESULTS A readiness table: the Custom ProfileXML profile, the Trusted Root CA profile, and the SCEP machine-cert profile each Found/Missing with assignment state (and whether the ProfileXML is assigned to a DEVICE group), plus the pilot device-group membership count. All green = ready to pilot; any Missing/Unassigned = fix before deploying (see 02-requirements.md / 03-implementation.md). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. VPN gateway head-end internals deferred to vendor docs. >

## AOVDT-Remediation.ps1

**Purpose:** Safe remediation for the Always On VPN device-tunnel deployment. Finds devices where the Custom ProfileXML profile ('AOVPN – Device Tunnel – ProfileXML – Windows') reported an error and finds expected profiles that are NOT assigned to the pilot/production DEVICE group, then (1) triggers a device sync to retry errored profile applications and (2) adds the missing assignment to the target DEVICE group. Defaults to preview only (-WhatIf): it PRINTS what it would do and changes nothing until you re-run with -WhatIf:$false. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\AOVDT-Remediation.ps1 Apply changes: powershell -ExecutionPolicy Bypass -File .\AOVDT-Remediation.ps1 -WhatIf:$false Required Microsoft Graph scopes (delegated): DeviceManagementConfiguration.ReadWrite.All DeviceManagementManagedDevices.ReadWrite.All (for device sync) Group.Read.All EXPECTED RESULTS A plan listing each remediation action (retry-sync device X; assign profile Y to the DEVICE group). In preview every line is prefixed [WHATIF]. With -WhatIf:$false each action runs and prints [DONE]/[FAILED]. Head-end / IKEv2 / CA-trust problems are NOT fixed here — see 06-troubleshooting.md. .NOTES White-label — the repeatable way. Write actions gated behind -WhatIf (default ON). VPN gateway head-end internals deferred to vendor docs. >

**How to run:** Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\AOVDT-Remediation.ps1 Apply changes: powershell -ExecutionPolicy Bypass -File .\AOVDT-Remediation.ps1 -WhatIf:$false Required Microsoft Graph scopes (delegated): DeviceManagementConfiguration.ReadWrite.All DeviceManagementManagedDevices.ReadWrite.All (for device sync) Group.Read.All EXPECTED RESULTS A plan listing each remediation action (retry-sync device X; assign profile Y to the DEVICE group). In preview every line is prefixed [WHATIF]. With -WhatIf:$false each action runs and prints [DONE]/[FAILED]. Head-end / IKEv2 / CA-trust problems are NOT fixed here — see 06-troubleshooting.md. .NOTES White-label — the repeatable way. Write actions gated behind -WhatIf (default ON). VPN gateway head-end internals deferred to vendor docs. >

## AOVDT-Report.ps1

**Purpose:** Read-only fleet coverage report for the Windows Always On VPN device-tunnel deployment. For the Custom OMA-URI ProfileXML profile and each certificate profile (Trusted Root CA, SCEP machine cert), pulls Intune apply-status counts (succeeded, error/conflict, pending, not applicable) via deviceStatusOverview and prints a coverage summary plus an optional CSV. Use it to prove pilot coverage and to watch drift during expansion. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AOVDT-Report.ps1 Optional: -CsvPath .\aovdt-coverage.csv to export per-object rows. Required Microsoft Graph scopes (delegated, read-only): DeviceManagementConfiguration.Read.All DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A table per profile with Succeeded, Failed, Pending, NotApplicable counts and a coverage percentage, plus an overall figure. Target: pilot 100%; expansion >= your threshold (e.g. 95%). Rows with Failed>0 flag where to run remediation. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. VPN gateway head-end internals deferred to vendor docs. >

**How to run:** powershell -ExecutionPolicy Bypass -File .\AOVDT-Report.ps1 Optional: -CsvPath .\aovdt-coverage.csv to export per-object rows. Required Microsoft Graph scopes (delegated, read-only): DeviceManagementConfiguration.Read.All DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A table per profile with Succeeded, Failed, Pending, NotApplicable counts and a coverage percentage, plus an overall figure. Target: pilot 100%; expansion >= your threshold (e.g. 95%). Rows with Failed>0 flag where to run remediation. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. VPN gateway head-end internals deferred to vendor docs. >

## AOVDT-Set-ScopeTargeting.ps1

**Purpose:** Creates the assignment target groups for the Always On VPN device-tunnel deployment — 'PILOT – AOVPN Device Tunnel – Devices' and (optionally) 'PROD – AOVPN Device Tunnel – Devices' — as assignable security groups, and adds the DEVICES you supply (by device display name). Because the device-tunnel ProfileXML deploys in device/SYSTEM context, these groups contain DEVICE objects, not users. Idempotent: if a group already exists it is reused, and devices already present are skipped. Defaults to preview only (-WhatIf); re-run with -WhatIf:$false to actually create/populate. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\AOVDT-Set-ScopeTargeting.ps1 -PilotDevices DEVICE01,DEVICE02 Apply: powershell -ExecutionPolicy Bypass -File .\AOVDT-Set-ScopeTargeting.ps1 -PilotDevices DEVICE01 -WhatIf:$false Required Microsoft Graph scopes (delegated): Group.ReadWrite.All GroupMember.ReadWrite.All Device.Read.All EXPECTED RESULTS The pilot group (and prod group if -CreateProd) exist and contain the listed DEVICES. In preview each create/add is prefixed [WHATIF]; with -WhatIf:$false each prints [DONE]/[FAILED]. These exact group names are what the ProfileXML + certificate profile assignments target in 03-implementation.md. .NOTES White-label — the repeatable way. Write actions gated behind -WhatIf (default ON). VPN gateway head-end internals

deferred to vendor docs. >

**How to run:** Preview: powershell -ExecutionPolicy Bypass -File .\AOVDT-Set-ScopeTargeting.ps1 -PilotDevices DEVICE01,DEVICE02 Apply: powershell -ExecutionPolicy Bypass -File .\AOVDT-Set-ScopeTargeting.ps1 -PilotDevices DEVICE01 -WhatIf:$false Required Microsoft Graph scopes (delegated): Group.ReadWrite.All GroupMember.ReadWrite.All Device.Read.All EXPECTED RESULTS The pilot group (and prod group if -CreateProd) exist and contain the listed DEVICES. In preview each create/add is prefixed [WHATIF]; with -WhatIf:$false each prints [DONE]/[FAILED]. These exact group names are what the ProfileXML + certificate profile assignments target in 03-implementation.md. .NOTES White-label — the repeatable way. Write actions gated behind -WhatIf (default ON). VPN gateway head-end internals deferred to vendor docs. >

## AOVDT-Validation.ps1

**Purpose:** Local, read-only validation run ON a pilot Windows device. Confirms the 'AOVPN Device Tunnel' VPN profile is present at the machine level (Get-VpnConnection -AllUserConnection), reports its ConnectionStatus, checks that a valid machine certificate with a Client Authentication EKU is in Cert:\LocalMachine\My, and best-effort reports whether DeviceTunnel/AlwaysOn are set. Prints a clear PASS/FAIL so a pilot device can be counted as validated. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\AOVDT-Validation.ps1 Optional: -GatewayFqdn vpn.your-company.com to check the profile targets a specific gateway. Run in an elevated (admin) PowerShell so LocalMachine store + rasphone are readable. No Microsoft Graph scopes needed — this inspects the local device only. EXPECTED RESULTS A checklist: device-tunnel profile present (yes/no + status), machine certificate present (yes/no, Client Auth EKU), DeviceTunnel/AlwaysOn best-effort. Overall PASS when the 'AOVPN Device Tunnel' profile exists AND a valid machine cert with Client Auth EKU is present; otherwise FAIL with the missing item. .NOTES White-label — the repeatable way. Read-only local inspection. VPN gateway head-end internals deferred to vendor docs. >

**How to run:** powershell -ExecutionPolicy Bypass -File .\AOVDT-Validation.ps1 Optional: -GatewayFqdn vpn.your-company.com to check the profile targets a specific gateway. Run in an elevated (admin) PowerShell so LocalMachine store + rasphone are readable. No Microsoft Graph scopes needed — this inspects the local device only. EXPECTED RESULTS A checklist: device-tunnel profile present (yes/no + status), machine certificate present (yes/no, Client Auth EKU), DeviceTunnel/AlwaysOn best-effort. Overall PASS when the 'AOVPN Device Tunnel' profile exists AND a valid machine cert with Client Auth EKU is present; otherwise FAIL with the missing item. .NOTES White-label — the repeatable way. Read-only local inspection. VPN gateway head-end internals deferred to vendor docs. >